Connecting PIC to GitHub and Neon

This guide takes the code on this computer to a GitHub repository and a Neon PostgreSQL database. Each push to `main` then updates the database automatically.

Where things stand:

- A local git repository is set up with one commit on `main`.
- The workflow file `.github/workflows/neon.yml` is in place.
- Nothing has been pushed or loaded into Neon yet.

Commands are for **PowerShell** in the project folder `C:\Users\SSINFOTECH\Desktop\supplier-capability-twin`.

Contents

1. Create the Neon project
2. Put the connection string in backend/.env
3. Load PIC into Neon from this computer
4. Create the GitHub repository and push
5. Add the GitHub secrets
6. Connect Neon to GitHub (preview branches)
7. Check that it works
8. Run the app against Neon
9. Troubleshooting

---

1. Create the Neon project

1. Sign in at https://console.neon.tech and click **New project**.
2. Fill in the project:

Field	Value
Name	`alphahelion-pic`
Postgres version	**16**. The migrations and tests are verified on 16.
Region	The region closest to your users. India data residency (Technical Document §14, DPDP) is a decision for you to make: check Neon's current region list.
Database name	`neondb` (default)

3. Click **Create project**.

2. Put the connection string in backend/.env

1. In Neon, open the project and click **Connect**.
2. Choose branch `main`, database `neondb` and role `neondb_owner`.
3. **Turn connection pooling off**, so that you get the *direct* host: there is no `-pooler` in the host name. The migrations need the direct host.
4. Copy the connection string. It looks like:

   ```
   postgresql://neondb_owner:<password>@ep-xxxx-xxxx.<region>.aws.neon.tech/neondb?sslmode=require
   ```

5. Create `backend\.env` from the example:

   ```
   Copy-Item backend\.env.example backend\.env
   ```

6. Generate the two keys. Run each command and paste its output into `backend\.env`:

   ```
   backend\venv\Scripts\python -c "import secrets;print(secrets.token_urlsafe(48))"
   backend\venv\Scripts\python -c "import
   secrets,base64;print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode())"
   ```

   The first output is `PIC_JWT_SECRET` and the second is `PIC_FIELD_KEY`.

7. Edit `backend\.env` (`notepad backend\.env`) so that it contains:

   ```
   DATABASE_URL=postgresql://neondb_owner:<password>@ep-xxxx.<region>.aws.neon.tech/neondb?sslmode=require
   PIC_JWT_SECRET=<first output>
   PIC_FIELD_KEY=<second output>
   PIC_SEED_PASSWORD=<a password for the demo users, at least 12 characters>
   PIC_ENV=dev
   ```

   Keep `PIC_FIELD_KEY` safe and never change it after the first load. It encrypts bank accounts, GST numbers and sealed bids. Data encrypted with one key cannot be read with another. Store a copy in your password manager.

`backend\.env` is git-ignored and never pushed.

3. Load PIC into Neon from this computer

```
cd backend
venv\Scripts\python -m app.pic.tools.neon_setup
```

It takes about 5–10 minutes and prints four steps:

Step	What happens
1/4 PIC migrations	Migrations 0001–0003 run: 101 tables in schema `pic`, the `pic_app` role, row-level security and the append-only audit
2/4 PIC seed	The Axion demo tenant (AXN) is loaded: the signature case, operating cases 1–8 and the ERP spend history
3/4 Legacy data	The Docker `supplier_twin` snapshot is copied into `public` and migrated into tenants GMC and DBY. This also writes `docs/PIC_Legacy_Migration_Report.md`.
4/4 Verification	Row counts, RLS coverage and audit chains are checked

The last lines should read:

```
    audit chain AXN: valid (…); pending events 0
   audit chain DBY: valid (…); pending events 0
   audit chain GMC: valid (…); pending events 0
OK
```

You can safely run it again; it never duplicates data. To only re-check:

```
venv\Scripts\python -m app.pic.tools.neon_setup --verify-only
```

If the legacy amounts are in USD, add `--fx-rate <rate>` on the **first** run, for example `--fx-rate 84.25`.

4. Create the GitHub repository and push

1. On https://github.com/new:
    - Name: `supplier-capability-twin`.
    - Visibility: **Private**.
    - **Do not** add a README, .gitignore or licence, so that the repository stays empty.
2. Click **Create repository** and copy its HTTPS URL.
3. In PowerShell, from the project folder:

    ```
    cd C:\Users\SSINFOTECH\Desktop\supplier-capability-twin
    git add -A
    git commit -m "Read PIC_SEED_PASSWORD from backend/.env; GitHub and Neon setup guide"
    git remote add origin https://github.com/<your-account>/supplier-capability-twin.git
    git push -u origin main
    ```

    The commit step picks up the small changes made after the first commit. If git says "nothing to commit", carry on.
4. On the first push a **GitHub sign-in window** opens (Git Credential Manager). Sign in and authorise. Later pushes reuse the stored credential.

Check on GitHub that you see `backend/`, `frontend/`, `docs/` and `.github/workflows/`, and that there is **no** `backend/.env`.

5. Add the GitHub secrets

In the repository go to **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Add the four secrets below with **exactly the same values as in** `backend\.env`:

Secret	Value
`DATABASE_URL`	The direct Neon connection string
`PIC_JWT_SECRET`	Same as in `.env`
`PIC_FIELD_KEY`	Same as in `.env` (never a new one)
`PIC_SEED_PASSWORD`	Same as in `.env`

The workflow refuses to run without `PIC_SEED_PASSWORD`, so that no generated password can appear in a public log.

Optional: under **Settings** → **Environments**, create `neon-production` and add yourself as a *required reviewer*. Every deployment to the production database then waits for your approval.

6. Connect Neon to GitHub (preview branches)

This step gives every pull request its own copy of the database, which is deleted when the PR closes.

1. In the Neon Console open the project, then **Integrations** → **GitHub** → **Add**, and authorise the **Neon** GitHub app.
2. Select the repository `supplier-capability-twin` and click **Connect**.
3. Neon adds the **secret** `NEON_API_KEY` and the **variable** `NEON_PROJECT_ID` to the repository by itself. Check this under **Settings** → **Secrets and variables** → **Actions**, on the Secrets and Variables tabs.

7. Check that it works

On GitHub:

1. Open the **Actions** tab. After the push you should see two workflows:
   - **CI** runs the backend tests (against its own temporary Postgres, never Neon), the frontend type check and the build;
   - **Neon database** has the job `deploy-main`, which runs `neon_setup` against Neon. Since step 3 has already loaded everything, it should only verify and report `OK`.
2. If `deploy-main` ran before you added the secrets, it fails with "Missing repository secrets". Add them (step 5) and click **Re-run jobs**.
3. Test a preview branch:
   - create a branch, change any file under `backend/`, and open a pull request;
   - the job `preview-branch` creates the Neon branch `preview/pr-<n>` and loads it;
   - when you close or merge the PR, `delete-preview-branch` removes it.

In the Neon Console, open **Tables** on branch `main`, schema `pic`. You should see 101 tables. `pic.tenant` should have three rows: AXN, GMC and DBY.

8. Run the app against Neon

The API reads `backend\.env`, so it now uses Neon.

```
 # terminal 1 – API
cd backend
venv\Scripts\python -m uvicorn app.main:app --host 127.0.0.1 --port 8080

# terminal 2 – worker (outbox, timers, ERP retries)
cd backend
venv\Scripts\python -m app.pic.worker

# terminal 3 – web
cd frontend
npm run dev
```

Open http://localhost:3000/pic. Sign in with organisation **AXN** (or GMC / DBY), a user such as `cpo@axion.example`, and your `PIC_SEED_PASSWORD`.

For a hosted deployment, set the same four values as environment variables on the server. The API may use the **pooled** host (`-pooler`); migrations and `neon_setup` must keep the direct host.

9. Troubleshooting

Symptom	Cause and fix
`password authentication failed`	The password in the URL has special characters. Copy the string again from Neon's **Connect** dialog, which encodes it.
`SSL connection is required`	Add `?sslmode=require` at the end of `DATABASE_URL`.
Migrations hang or fail with an advisory-lock or prepared-statement error	You used the `-pooler` host. Switch to the direct host for `neon_setup`.
`permission denied to create role`	Use the project owner role (`neondb_owner`), not a role you created yourself.
`could not decrypt` / `InvalidToken` when opening suppliers	`PIC_FIELD_KEY` is different from the key used at first load. Restore the original key.
Push rejected: `fetch first` / `non-fast-forward`	The GitHub repository was not empty. Run `git pull origin main --allow-unrelated-histories`, resolve, and push again; or recreate the repository empty.
Sign-in window does not appear on push	Run `git credential-manager configure`, or create a fine-grained personal access token (repository Contents: read/write) and use it as the password.
`deploy-main` fails with `Missing repository secrets`	Add the secrets from step 5 and re-run the job.
`preview-branch` is skipped	`NEON_PROJECT_ID` is not set. Complete step 6.
Tests must never run on Neon	They refuse to: `tests/pic/conftest.py` stops if the URL contains `neon.tech`. CI always uses its own Postgres container.